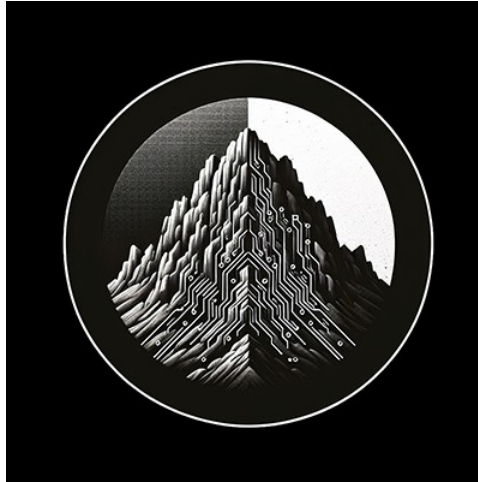




RTL Design Sherpa

RTL AMBA AXI4-Lite Module Reference — Rev 0.90

July 2026

Table of Contents

1 AXIL4 Clock-Gated Variants Guide.....	9
1.1 Overview.....	9
1.1.1 Available Clock-Gated Modules.....	9
1.1.2 Key Features.....	10
1.2 Additional Parameters (All _cg Modules).....	10
1.3 Additional Ports (All _cg Modules).....	10
1.3.1 Clock Gating Configuration.....	10
1.3.2 Clock Gating Status.....	10
1.4 Usage Example.....	11
1.5 Clock Gating Behavior.....	11
1.6 Configuration Guidelines.....	12
1.6.1 Idle Count Selection.....	12
1.6.2 When to Use Clock Gating.....	12
1.7 Power Savings Estimates.....	12
1.7.1 Typical Savings (AXI4-Lite Register Access).....	12
1.8 Complete Documentation.....	13
1.9 Related Documentation.....	13
1.9.1 Base Modules.....	13
1.9.2 Architecture.....	13
1.10 Navigation.....	14
2 AXIL4 Master Read Interface (Clock-Gated).....	14
2.1 Quick Reference.....	14

2.2 Summary.....	14
2.3 Common Parameters.....	14
2.4 Quick Usage.....	15
2.5 Documentation.....	15
2.6 Navigation.....	15
3 AXIL4 Master Read.....	16
3.1 Overview.....	16
3.1.1 Key Features.....	16
3.2 Parameters.....	16
3.2.1 Derived Parameters.....	17
3.3 Port Groups.....	17
3.3.1 Clock and Reset.....	17
3.3.2 Frontend AXI4-Lite Read Interface (Input Side).....	17
3.3.3 Backend AXI4-Lite Master Interface (Output Side).....	17
3.3.4 Status Outputs.....	18
3.4 Signal Flow.....	19
3.4.1 Read Transaction Sequence.....	19
3.5 Usage Example.....	19
3.5.1 Basic Integration.....	19
3.5.2 Register Read Example.....	20
3.6 Design Notes.....	20
3.6.1 AXI4-Lite Protocol Constraints.....	20
3.6.2 Buffer Depth Selection.....	21
3.6.3 Busy Signal.....	21

3.6.4 PROT Encoding.....	21
3.7 Performance Characteristics.....	22
3.7.1 Latency.....	22
3.7.2 Throughput.....	22
3.7.3 Resource Usage.....	22
3.8 Related Modules.....	22
3.8.1 Core Modules.....	22
3.8.2 Monitor Modules.....	22
3.8.3 Clock-Gated Variants.....	22
3.8.4 Supporting Infrastructure.....	23
3.9 References.....	23
3.9.1 Specifications.....	23
3.9.2 Documentation.....	23
3.9.3 Source Code.....	23
3.10 Navigation.....	23
4 AXIL4 Master Write Interface (Clock-Gated).....	23
4.1 Quick Reference.....	23
4.2 Summary.....	24
4.3 Common Parameters.....	24
4.4 Quick Usage.....	24
4.5 Documentation.....	25
4.6 Navigation.....	25
5 AXIL4 Master Write.....	25
5.1 Overview.....	25

5.1.1 Key Features.....	25
5.2 Parameters.....	26
5.2.1 Derived Parameters.....	26
5.3 Port Groups.....	27
5.3.1 Clock and Reset.....	27
5.3.2 Frontend AXI4-Lite Write Interface (Input Side).....	27
5.3.3 Backend AXI4-Lite Master Interface (Output Side).....	27
5.3.4 Status Outputs.....	28
5.4 Signal Flow.....	30
5.4.1 Write Transaction Sequence.....	30
5.5 Usage Example.....	30
5.5.1 Basic Integration.....	30
5.5.2 Register Write Example.....	31
5.5.3 Partial Write Example (Byte Strobes).....	32
5.6 Design Notes.....	32
5.6.1 AXI4-Lite Protocol Constraints.....	32
5.6.2 Buffer Depth Selection.....	32
5.6.3 Write Strobe (WSTRB) Usage.....	33
5.6.4 Busy Signal.....	33
5.7 Performance Characteristics.....	33
5.7.1 Latency.....	33
5.7.2 Throughput.....	33
5.7.3 Resource Usage.....	34
5.8 Related Modules.....	34

5.8.1 Core Modules.....	34
5.8.2 Monitor Modules.....	34
5.8.3 Clock-Gated Variants.....	34
5.8.4 Supporting Infrastructure.....	34
5.9 References.....	34
5.9.1 Specifications.....	34
5.9.2 Documentation.....	34
5.9.3 Source Code.....	35
5.10 Navigation.....	35
6 AXIL4 Slave Read Interface (Clock-Gated).....	35
6.1 Quick Reference.....	35
6.2 Summary.....	35
6.3 Common Parameters.....	36
6.4 Quick Usage.....	36
6.5 Documentation.....	36
6.6 Navigation.....	37
7 AXIL4 Slave Read.....	37
7.1 Overview.....	37
7.1.1 Key Features.....	37
7.2 Parameters.....	37
7.3 Port Groups.....	38
7.3.1 Slave Interface (Input Side from Interconnect).....	38
7.3.2 Backend Interface (Output Side to Memory/Logic).....	38
7.3.3 Status Outputs.....	38

7.4 Usage Example.....	39
7.5 Design Notes.....	39
7.6 Related Modules.....	40
7.7 Navigation.....	40
8 AXIL4 Slave Write Interface (Clock-Gated).....	40
8.1 Quick Reference.....	40
8.2 Summary.....	41
8.3 Common Parameters.....	41
8.4 Quick Usage.....	41
8.5 Documentation.....	42
8.6 Navigation.....	42
9 AXIL4 Slave Write.....	42
9.1 Overview.....	42
9.1.1 Key Features.....	42
9.2 Parameters.....	43
9.3 Port Groups.....	43
9.3.1 Slave Interface (Input Side from Interconnect).....	43
9.3.2 Backend Interface (Output Side to Memory/Logic).....	43
9.3.3 Status Outputs.....	44
9.4 Usage Example.....	44
9.5 Design Notes.....	45
9.6 Related Modules.....	45
9.7 Navigation.....	46
10 AXIL4 (AXI4-Lite) Modules.....	46

10.1 Overview.....	46
10.1.1 Key Features.....	46
10.2 Module Categories.....	46
10.2.1 Core Master/Slave Modules.....	46
10.2.2 Monitor Modules (Verification).....	47
10.2.3 Clock-Gated Variants.....	47
10.3 AXI4-Lite Protocol Overview.....	47
10.3.1 Simplified vs Full AXI4.....	47
10.3.2 Channel Architecture.....	48
10.3.3 Key Signals (Simplified).....	48
10.4 Quick Start.....	48
10.4.1 Basic Read Master.....	48
10.4.2 Basic Write Slave.....	49
10.4.3 Monitor Integration.....	49
10.5 Testing.....	50
10.6 Design Patterns.....	51
10.6.1 Pattern 1: Buffered Master/Slave.....	51
10.6.2 Pattern 2: Monitor Integration.....	51
10.6.3 Pattern 3: Clock Gating.....	51
10.7 Performance Characteristics.....	51
10.7.1 Throughput.....	51
10.7.2 Latency.....	51
10.7.3 Resource Usage.....	52
10.8 Related Documentation.....	52

10.8.1 Protocol Specifications.....	52
10.8.2 RTL Design Sherpa Documentation.....	52
10.8.3 Configuration and Integration.....	52
10.8.4 Source Code.....	52
10.9 Design Notes.....	53
10.9.1 When to Use AXI4-Lite.....	53
10.9.2 Buffer Depth Guidelines.....	53
10.10 Navigation.....	53

1 AXIL4 Clock-Gated Variants Guide

Location: rtl/amba/axil4/*_cg.sv **Status:** ✓ Production Ready

1.1 Overview

All AXIL4 modules have clock-gated (_cg) variants that add power management through dynamic clock gating. Identical architecture to [AXI4 Clock Gating](#) but optimized for AXI4-Lite's simpler protocol.

1.1.1 Available Clock-Gated Modules

Module	Base Module	Description
axil4_master_rd_cg	axil4_master_rd	Clock-gated read master
axil4_master_wr_cg	axil4_master_wr	Clock-gated write master
axil4_slave_rd_cg	axil4_slave_rd	Clock-gated read slave
axil4_slave_wr_cg	axil4_slave_wr	Clock-gated write slave
axil4_master_rd_mon_cg	axil4_master_rd_mon	Clock-gated read master + monitor
axil4_master_wr_mon_cg	axil4_master_wr_mon	Clock-gated write master + monitor

Module	Base Module	Description
axil4_slave_rd_mon_cg	axil4_slave_rd_mon	Clock-gated read slave + monitor
axil4_slave_wr_mon_cg	axil4_slave_wr_mon	Clock-gated write slave + monitor

1.1.2 Key Features

- ✓ **Dynamic Clock Gating:** Automatic clock disable during idle
- ✓ **Configurable Idle Threshold:** Programmable idle count before gating
- ✓ **Full Functional Equivalence:** Identical to base modules
- ✓ **Status Monitoring:** Real-time gating status and clock count
- ✓ **Test Mode Support:** Bypass for scan testing
- ✓ **Zero Performance Impact:** Immediate ungating on activity

1.2 Additional Parameters (All _cg Modules)

Parameter	Type	Default	Description
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter (max idle = $2^N - 1$ cycles)

All other parameters identical to base module.

1.3 Additional Ports (All _cg Modules)

1.3.1 Clock Gating Configuration

Port	Direction	Width	Description
cfg_cg_enable	Input	1	Enable clock gating (0=disabled, 1=enabled)
cfg_cg_idle_count	Input	CG_IDLE_COUNT_WIDTH	Idle cycles before gating

1.3.2 Clock Gating Status

Port	Direction	Width	Description
cg_gating	Output	1	Clock currently gated

Port	Direction	Width	Description
cg_clk_count	Output	32	(1=gated, 0=running) Cumulative gated clock cycles

All other ports identical to base module.

1.4 Usage Example

```
axil4_master_rd_cg #(
    // Base parameters
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AR(2),
    .SKID_DEPTH_R(4),

    // Clock gating
    .CG_IDLE_COUNT_WIDTH(4) // Up to 15 idle cycles
) u_axil_rd_cg (
    .aclk(axi_clk),
    .aresetn(axi_resetn),

    // Clock gating configuration
    .cfg_cg_enable(1'b1), // Enable gating
    .cfg_cg_idle_count(4'd5), // Gate after 5 idle cycles

    // AXIL signals (identical to base module)
    .fub_axil_araddr(cpu_araddr),
    // ... rest of AR/R signals ...

    // Clock gating status
    .cg_gating(rd_clk_gated),
    .cg_clk_count(rd_gated_cycles)
);
```

1.5 Clock Gating Behavior

Gating Conditions (All Must Be True): 1. Interface idle (no valid/ready handshakes) 2. Idle for $\geq$ cfg_cg_idle_count consecutive cycles 3. Gating enabled (cfg_cg_enable = 1)

Ungating Conditions (Any Triggers Immediate Ungating): 1. Any *valid signal asserted on any channel 2. Configuration change

Ungating Latency: 0 cycles (immediate)

1.6 Configuration Guidelines

1.6.1 Idle Count Selection

Aggressive Gating (Register Access):

```
.cfg_cg_idle_count(4'd1)    // Gate after 1 idle cycle  
// AXI4-Lite typical: Burst register access with idle gaps
```

Moderate Gating (Balanced):

```
.cfg_cg_idle_count(4'd5)    // Gate after 5 idle cycles  
// General purpose configuration
```

Conservative Gating (Low Latency):

```
.cfg_cg_idle_count(4'd10)   // Gate after 10 idle cycles  
// Critical control paths, minimal overhead
```

Disable Gating:

```
.cfg_cg_enable(1'b0)        // Clock gating disabled  
// 100% utilization or ultra-low-latency paths
```

1.6.2 When to Use Clock Gating

✓ **Recommended For:** - Register access patterns (idle between accesses) - Low-duty-cycle control interfaces - Power-constrained systems

✗ **Not Recommended For:** - 100% utilization scenarios - Ultra-low-latency critical paths - Continuous register polling

1.7 Power Savings Estimates

1.7.1 Typical Savings (AXI4-Lite Register Access)

Traffic Pattern	Duty Cycle	Idle Count	Power Savings
Burst register reads	20%	1	35-40%

Traffic Pattern	Duty Cycle	Idle Count	Power Savings
Sporadic writes	10%	1	40-45%
Periodic polling	50%	5	20-25%
High activity	80%	5	5-10%

Note: AXI4-Lite often has better power savings than AXI4 due to: - Single-beat transactions (more idle gaps) - Simpler control (faster idle detection) - Register access patterns (bursty by nature)

1.8 Complete Documentation

For full clock gating details, see: - [AXI4 Clock Gating Guide](#) - Complete reference with: - Detailed gating behavior and state machine - Power savings calculations and examples - Dynamic configuration patterns - Test mode support - Synthesis considerations

AXI4-specific notes: - Same architecture as AXI4 clock gating - Typically better power savings (bursty register access) - Simpler configuration (fewer outstanding transactions)

1.9 Related Documentation

1.9.1 Base Modules

- [axil4_master_rd](#) - Base read master
- [axil4_master_wr](#) - Base write master
- [axil4_slave_rd](#) - Base read slave
- [axil4_slave_wr](#) - Base write slave
- [Monitor modules](#) - Base monitor modules

1.9.2 Architecture

- [AXI4 Clock Gating Guide](#) - Complete reference
 - [RTLamba Overview](#) - Complete AMBA subsystem
 - [AXI4 Index](#) - AXI4 module index
-

Last Updated: 2025-10-20

1.10 Navigation

- [← Back to AXIL4 Index](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

2 AXIL4 Master Read Interface (Clock-Gated)

Module: axil4_master_rd_cg.sv **Base Module:** [axil4_master_rd](#) **Location:** rtl/amba/axil4/ **Status:** ✓ Production Ready

2.1 Quick Reference

This is the **clock-gated variant** of [axil4_master_rd](#).

For complete clock-gating documentation, usage examples, and configuration guidelines, see:

→ [Clock-Gated Variants Guide](#)

2.2 Summary

The axil4_master_rd_cg module adds power optimization to axil4_master_rd through activity-based clock gating:

- ✓ **Same Functionality:** 100% equivalent to base module
 - ✓ **Power Savings:** 25-70% depending on traffic utilization
 - ✓ **Configurable:** Idle threshold, gating domains, enable/disable
 - ✓ **Zero Overhead When Disabled:** ENABLE_CLOCK_GATING=0 → identical to base
-

2.3 Common Parameters

In addition to all [axil4_master_rd](#) parameters:

Parameter	Default	Description
ENABLE_CLOCK_GATING	1	Master enable (0=disable,

Parameter	Default	Description
CG_IDLE_CYCLES	8	identical to base) Cycles before clock gating activates
CG_GATE_*	1	Domain-specific gating enables

2.4 Quick Usage

```
axil4_master_rd_cg #(
    // Base module parameters (see axil4_master_rd.md)
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),

    // Clock gating (see CG guide for details)
    .ENABLE_CLOCK_GATING(1),
    .CG_IDLE_CYCLES(8)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),
    // ... all other ports same as axil4_master_rd
);
```

2.5 Documentation

- **Base Module Functionality:** [axil4_master_rd.md](#)
- **Clock Gating Guide:** [clock_gated_variants.md](#)
- **Detailed CG Examples:**
 - [axil4_master_rd_mon_cg.md](#) (AXI4 monitor)
 - [axil4_master_rd_mon_cg.md](#) (AXIL4 monitor)
 - [apb_slave_cg.md](#) (APB interface)

2.6 Navigation

- [← Back to AXIL4 Index](#)
- [← Back to RTLAmba Index](#)

- [← Back to Main Documentation Index](#)

3 AXIL4 Master Read

Module: axil4_master_rd.sv **Location:** rtl/amba/axil4/ **Status:** ✓ Production Ready

3.1 Overview

The AXIL4 Master Read module provides a buffered AXI4-Lite read interface for master devices. AXI4-Lite is a simplified subset of AXI4 designed for control register access with reduced signal count and single-beat transactions only.

3.1.1 Key Features

- ✓ **AXI4-Lite Protocol:** Simplified read-only interface (AR and R channels)
- ✓ **Single-Beat Transactions:** No burst support (always 1 transfer)
- ✓ **Buffered Channels:** Configurable skid buffers for timing closure
- ✓ **Elastic Buffering:** Decouples frontend and backend timing
- ✓ **Activity Monitoring:** Busy signal for clock gating
- ✓ **Minimal Latency:** 1-cycle buffer overhead per channel

3.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	Address bus width (typically 32)
AXIL_DATA_WIDTH	int	32	Data bus width (32 or 64)
SKID_DEPTH_AR	int	2	AR channel buffer depth (2 ^N entries)
SKID_DEPTH_R	int	4	R channel buffer depth (2 ^N entries)

3.2.1 Derived Parameters

Parameter	Calculation	Description
AW	AXIL_ADDR_WIDTH	Internal address width alias
DW	AXIL_DATA_WIDTH	Internal data width alias
ARSize	AW+3	AR packet size (addr + prot)
RSize	DW+2	R packet size (data + resp)

3.3 Port Groups

3.3.1 Clock and Reset

Port	Direction	Width	Description
aclk	Input	1	AXI clock
aresetn	Input	1	Active-low asynchronous reset

3.3.2 Frontend AXI4-Lite Read Interface (Input Side)

Read Address Channel (AR): | Port | Direction | Width | Description | |——|———|——|
———| | fub_araddr | Input | AW | Read address | | fub_arprot | Input | 3 | Protection attributes | | fub_arvalid | Input | 1 | Address valid | | fub_arready | Output | 1 | Address ready (from buffer) |

Read Data Channel (R): | Port | Direction | Width | Description | |——|———|——|
———| | fub_rdata | Output | DW | Read data | | fub_rresp | Output | 2 | Read response (OKAY, SLVERR, DECERR) | | fub_rvalid | Output | 1 | Data valid (from buffer) | | fub_rready | Input | 1 | Data ready |

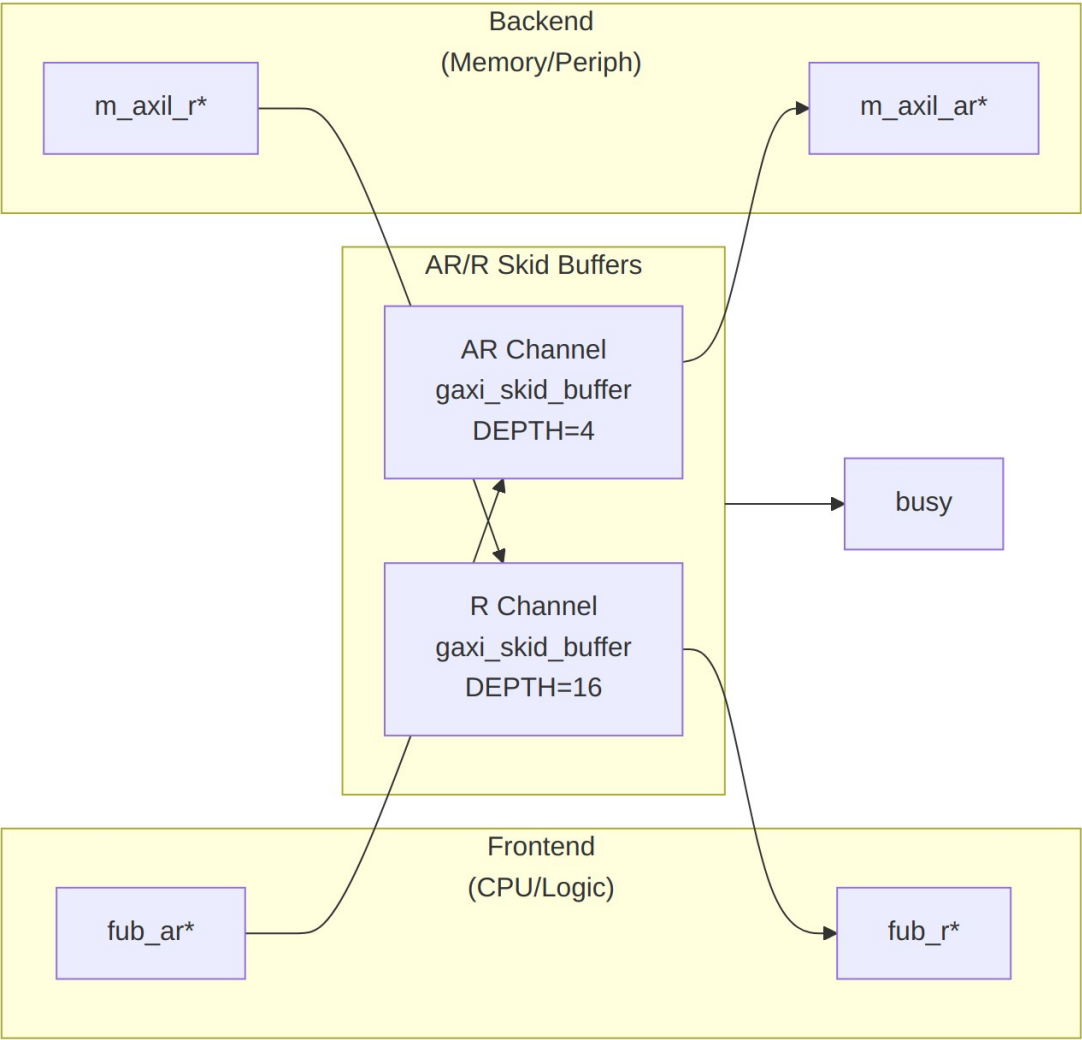
3.3.3 Backend AXI4-Lite Master Interface (Output Side)

Read Address Channel (AR): | Port | Direction | Width | Description | |——|———|——|
———| | m_axil_araddr | Output | AW | Read address (to slave) | | m_axil_arprot | Output | 3 | Protection attributes | | m_axil_arvalid | Output | 1 | Address valid (from buffer) | | m_axil_arready | Input | 1 | Address ready (from slave) |

Read Data Channel (R): | Port | Direction | Width | Description | |——|———|——|
 ————| | m_axil_rdata | Input | DW | Read data (from slave) | | m_axil_rresp | Input | 2
 | Read response | | m_axil_rvalid | Input | 1 | Data valid (from slave) | | m_axil_rready |
 Output | 1 | Data ready (to slave) |

3.3.4 Status Outputs

Port	Direction	Width	Description
busy	Output	1	Interface active (for clock gating)



Module Architecture

3.4 Signal Flow

3.4.1 Read Transaction Sequence

Cycle 1: fub_araddr=0x1000, fub_arvalid=1, fub_arprot=0
fub_arready=1 (buffer accepts)

Cycle 2: AR buffer forwards to backend:
m_axil_araddr=0x1000, m_axil_arvalid=1
m_axil_arready=1 (slave accepts)

Cycle 3-5: Slave processes read

Cycle 6: m_axil_rdata=0xDEADBEEF, m_axil_rresp=OKAY
m_axil_rvalid=1, m_axil_rready=1 (R buffer accepts)

Cycle 7: R buffer forwards to frontend:
fub_rdata=0xDEADBEEF, fub_rresp=OKAY
fub_rvalid=1, fub_rready=1 (frontend accepts)

3.5 Usage Example

3.5.1 Basic Integration

```
axil4_master_rd #(
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AR(2),      // 4 entries
    .SKID_DEPTH_R(4)       // 16 entries
) u_axil_master_rd (
    .aclk          (axi_clk),
    .aresetn       (axi_resetn),

    // Frontend interface (from CPU)
    .fub_araddr    (cpu_araddr),
    .fub_arprot    (cpu_arprot),
    .fub_arvalid   (cpu_arvalid),
    .fub_arready   (cpu_arready),

    .fub_rdata     (cpu_rdata),
    .fub_rresp     (cpu_rresp),
    .fub_rvalid    (cpu_rvalid),
```

```

.fub_rready      (cpu_rready),

// Backend interface (to peripheral/memory)
.m_axil_araddr   (periph_araddr),
.m_axil_arprot   (periph_arprot),
.m_axil_arvalid  (periph_arvalid),
.m_axil_arready  (periph_arready),

.m_axil_rdata    (periph_rdata),
.m_axil_rresp    (periph_rresp),
.m_axil_rvalid   (periph_rvalid),
.m_axil_rready   (periph_rready),

// Status
.busy            (rd_busy)
);

3.5.2 Register Read Example
// Read from control register at 0x1000
initial begin
    @(posedge aclk);
    fub_araddr = 32'h1000;
    fub_arprot = 3'b000; // Unprivileged, secure, data access
    fub_arvalid = 1'b1;

    @(posedge aclk);
    while (!fub_arready) @(posedge aclk); // Wait for buffer accept
    fub_arvalid = 1'b0;

    // Wait for read data
    fub_rready = 1'b1;
    @(posedge aclk);
    while (!fub_rvalid) @(posedge aclk);

    $display("Read data: 0x%08x, RRESP: %0d", fub_rdata, fub_rresp);
end

```

3.6 Design Notes

3.6.1 AXI4-Lite Protocol Constraints

Single-Beat Only: - No burst support (implicitly ARLEN=0, ARSIZE=log2(DW/8)) - Each AR transaction results in exactly 1 R transaction - No RLAST signal (always last)

No Out-of-Order: - No ARID/RID signals - Responses always in order - Simpler transaction tracking

Reduced Signals: - No ARCACHE, ARQOS, ARREGION, ARLOCK, ARSIZE, ARLEN, ARBURST - Only ARADDR and ARPROT for control

3.6.2 Buffer Depth Selection

AR Channel (Addresses): - **SKID_DEPTH_AR = 2** (4 entries): Typical for control registers - Increase if many back-to-back reads with slow slave response

R Channel (Data): - **SKID_DEPTH_R = 4** (16 entries): Accommodates pipeline latency - Increase for high-latency backends (DRAM, off-chip peripherals)

3.6.3 Busy Signal

Asserted when: - AR buffer has pending addresses (`int_ar_count > 0`) - R buffer has pending data (`int_r_count > 0`) - Frontend presenting new address (`fub_arvalid`) - Backend presenting new data (`m_axil_rvalid`)

Use for: - Clock gating control (see `axil4_master_rd_cg`) - Power management - Interface idle detection

3.6.4 PROT Encoding

ARPROT[2:0]	Privilege	Security	Type
3'b000	Unprivileged	Secure	Data
3'b001	Unprivileged	Secure	Instruction
3'b010	Unprivileged	Non-secure	Data
3'b011	Unprivileged	Non-secure	Instruction
3'b100	Privileged	Secure	Data
3'b101	Privileged	Secure	Instruction
3'b110	Privileged	Non-secure	Data
3'b111	Privileged	Non-secure	Instruction

Typical: 3'b000 (unprivileged secure data) for most peripherals

3.7 Performance Characteristics

3.7.1 Latency

Path	Cycles	Notes
Frontend → Backend (AR)	1	Skid buffer overhead
Backend → Frontend (R)	1	Skid buffer overhead
Total read latency	Slave latency + 2	AR + Slave + R

3.7.2 Throughput

Scenario	Rate	Notes
Back-to-back reads	1 read/cycle	If buffers not stalled
Typical peripheral	1 read/N cycles	N = slave response time

3.7.3 Resource Usage

Resource	Count	Notes
LUTs	~200	Approximate (32-bit)
FFs	~150	Buffer state + control
BRAM	0	Uses distributed RAM

3.8 Related Modules

3.8.1 Core Modules

- [axil4_master_wr](#) - Write master counterpart
- [axil4_slave_rd](#) - Slave read interface
- [axil4_slave_wr](#) - Slave write interface

3.8.2 Monitor Modules

- [axil4_master_rd_mon](#) - Master read with monitoring

3.8.3 Clock-Gated Variants

- [axil4_master_rd_cg](#) - Clock-gated version

3.8.4 Supporting Infrastructure

- **gaxi_skid_buffer** - Elastic buffering (rtl/amba/gaxi/)
-

3.9 References

3.9.1 Specifications

- ARM IHI 0022E: AMBA AXI and ACE Protocol Specification
- Chapter 4: AXI4-Lite (simplified subset)

3.9.2 Documentation

- [RTLamba Overview](#) - Complete AMBA subsystem
- [AXI4 Master Read](#) - Full AXI4 comparison

3.9.3 Source Code

- RTL: rtl/amba/axil4/axil4_master_rd.sv
 - Tests: val/amba/test_axil4_master_rd.py
 - Framework: bin/TBClasses/components/axil4/
-

Last Updated: 2025-10-20

3.10 Navigation

- [← Back to AXIL4 Index](#)
- [← Back to RTLamba Index](#)
- [← Back to Main Documentation Index](#)

4 AXIL4 Master Write Interface (Clock-Gated)

Module: axil4_master_wr_cg.sv **Base Module:** [axil4_master_wr](#) **Location:** rtl/amba/axil4/ **Status:** ✓ Production Ready

4.1 Quick Reference

This is the **clock-gated variant** of [axil4_master_wr](#).

For complete clock-gating documentation, usage examples, and configuration guidelines, see:

→ [Clock-Gated Variants Guide](#)

4.2 Summary

The `axil4_master_wr_cg` module adds power optimization to `axil4_master_wr` through activity-based clock gating:

- ✓ **Same Functionality:** 100% equivalent to base module
 - ✓ **Power Savings:** 25-70% depending on traffic utilization
 - ✓ **Configurable:** Idle threshold, gating domains, enable/disable
 - ✓ **Zero Overhead When Disabled:** `ENABLE_CLOCK_GATING=0` → identical to base
-

4.3 Common Parameters

In addition to all [axil4_master_wr](#) parameters:

Parameter	Default	Description
<code>ENABLE_CLOCK_GATING</code>	1	Master enable (0=disable, identical to base)
<code>CG_IDLE_CYCLES</code>	8	Cycles before clock gating activates
<code>CG_GATE_*</code>	1	Domain-specific gating enables

4.4 Quick Usage

```
axil4_master_wr_cg #(
    // Base module parameters (see axil4_master_wr.md)
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),

    // Clock gating (see CG guide for details)
    .ENABLE_CLOCK_GATING(1),
```



```

        .CG_IDLE_CYCLES(8)
    ) u_cg (
        .aclk(clk),
        .aresetn(rst_n),
        // ... all other ports same as axil4_master_wr
    );

```

4.5 Documentation

- **Base Module Functionality:** [axil4_master_wr.md](#)
 - **Clock Gating Guide:** [clock_gated_variants.md](#)
 - **Detailed CG Examples:**
 - [axi4_master_rd_mon_cg.md](#) (AXI4 monitor)
 - [axil4_master_rd_mon_cg.md](#) (AXIL4 monitor)
 - [apb_slave_cg.md](#) (APB interface)
-

4.6 Navigation

- [← Back to AXIL4 Index](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

5 AXIL4 Master Write

Module: `axil4_master_wr.sv` **Location:** `rtl/amba/axil4/` **Status:** ✓ Production Ready

5.1 Overview

The AXIL4 Master Write module provides a buffered AXI4-Lite write interface for master devices. AXI4-Lite is a simplified subset of AXI4 designed for control register access with reduced signal count and single-beat transactions only.

5.1.1 Key Features

- ✓ **AXI4-Lite Protocol:** Simplified write-only interface (AW, W, B channels)
- ✓ **Single-Beat Transactions:** No burst support (always 1 transfer)
- ✓ **Buffered Channels:** Configurable skid buffers for all 3 channels

- ✓ **Byte Strobes:**WSTRB support for partial word writes
- ✓ **Elastic Buffering:** Decouples frontend and backend timing
- ✓ **Activity Monitoring:** Busy signal for clock gating
- ✓ **Minimal Latency:** 1-cycle buffer overhead per channel

5.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	Address bus width (typically 32)
AXIL_DATA_WIDTH	int	32	Data bus width (32 or 64)
SKID_DEPTH_AW	int	2	AW channel buffer depth (2 ^N entries)
SKID_DEPTH_W	int	2	W channel buffer depth (2 ^N entries)
SKID_DEPTH_B	int	2	B channel buffer depth (2 ^N entries)

5.2.1 Derived Parameters

Parameter	Calculation	Description
AW	AXIL_ADDR_WIDTH	Internal address width alias
DW	AXIL_DATA_WIDTH	Internal data width alias
AWSize	AW+3	AW packet size (addr + prot)
WSize	DW+(DW/8)	W packet size (data + strb)
BSize	2	B packet size (resp only)

5.3 Port Groups

5.3.1 Clock and Reset

Port	Direction	Width	Description
acclk	Input	1	AXI clock
aresetn	Input	1	Active-low asynchronous reset

5.3.2 Frontend AXI4-Lite Write Interface (Input Side)

Write Address Channel (AW): | Port | Direction | Width | Description | |——|———|——|
———| | fub_awaddr | Input | AW | Write address | | fub_awprot | Input | 3 | Protection attributes | | fub_awvalid | Input | 1 | Address valid | | fub_awready | Output | 1 | Address ready (from buffer) |

Write Data Channel (W): | Port | Direction | Width | Description | |——|———|——|
———| | fub_wdata | Input | DW | Write data | | fub_wstrb | Input | DW/8 | Write strobes (byte lane enables) | | fub_wvalid | Input | 1 | Data valid | | fub_wready | Output | 1 | Data ready (from buffer) |

Write Response Channel (B): | Port | Direction | Width | Description | |——|———|——|
———| | fub_bresp | Output | 2 | Write response (OKAY, SLVERR, DECERR) | | fub_bvalid | Output | 1 | Response valid (from buffer) | | fub_bready | Input | 1 | Response ready |

5.3.3 Backend AXI4-Lite Master Interface (Output Side)

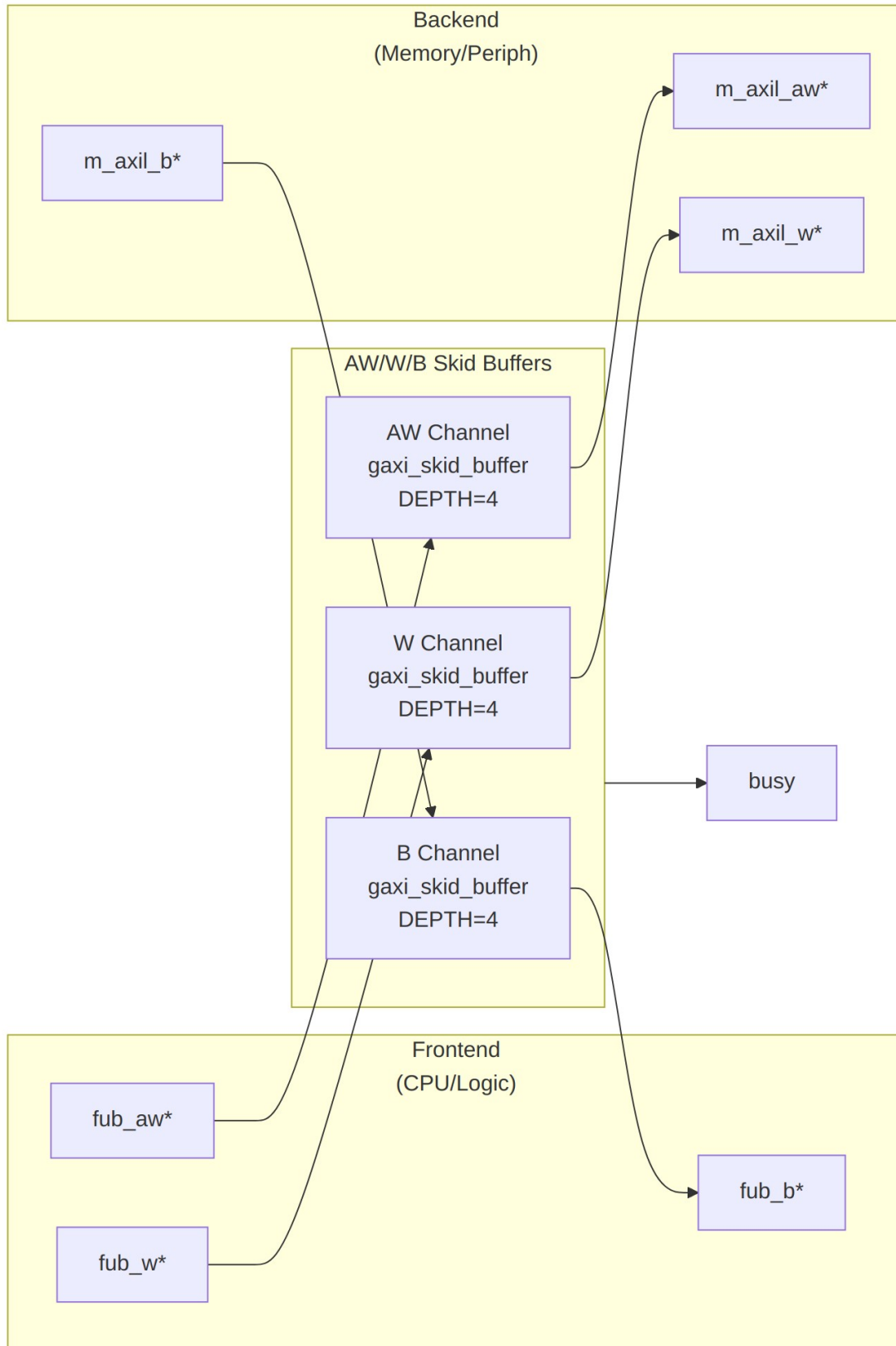
Write Address Channel (AW): | Port | Direction | Width | Description | |——|———|——|
———| | m_axil_awaddr | Output | AW | Write address (to slave) | | m_axil_awprot | Output | 3 | Protection attributes | | m_axil_awvalid | Output | 1 | Address valid (from buffer) | | m_axil_awready | Input | 1 | Address ready (from slave) |

Write Data Channel (W): | Port | Direction | Width | Description | |——|———|——|
———| | m_axil_wdata | Output | DW | Write data (to slave) | | m_axil_wstrb | Output | DW/8 | Write strobes | | m_axil_wvalid | Output | 1 | Data valid (from buffer) | | m_axil_wready | Input | 1 | Data ready (from slave) |

Write Response Channel (B): | Port | Direction | Width | Description | |——|———|——|
———| | m_axil_bresp | Input | 2 | Write response (from slave) | | m_axil_bvalid | Input | 1 | Response valid (from slave) | | m_axil_bready | Output | 1 | Response ready (to slave) |

5.3.4 Status Outputs

Port	Direction	Width	Description
busy	Output	1	Interface active (for clock gating)



5.4 Signal Flow

5.4.1 Write Transaction Sequence

Cycle 1: fub_awaddr=0x2000, fub_awvalid=1, fub_awprot=0
fub_wdata=0xCAFEBAFE, fub_wstrb=4'b1111, fub_wvalid=1
fub_awready=1, fub_wready=1 (buffers accept)

Cycle 2: Buffers forward to backend:
m_axil_awaddr=0x2000, m_axil_awvalid=1
m_axil_wdata=0xCAFEBAFE, m_axil_wstrb=4'b1111,
m_axil_wvalid=1
m_axil_awready=1, m_axil_wready=1 (slave accepts)

Cycle 3-5: Slave processes write

Cycle 6: m_axil_bresp=OKAY, m_axil_bvalid=1
m_axil_bready=1 (B buffer accepts)

Cycle 7: B buffer forwards to frontend:
fub_bresp=OKAY, fub_bvalid=1
fub_bready=1 (frontend accepts)

5.5 Usage Example

5.5.1 Basic Integration

```
axil4_master_wr #(
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AW(2),      // 4 entries
    .SKID_DEPTH_W(2),       // 4 entries
    .SKID_DEPTH_B(2)        // 4 entries
) u_axil_master_wr (
    .aclk      (axi_clk),
    .aresetn   (axi_resetn),

    // Frontend interface (from CPU)
    .fub_awaddr (cpu_awaddr),
    .fub_awprot (cpu_awprot),
    .fub_awvalid (cpu_awvalid),
```

```

.fub_awready      (cpu_awready),

.fub_wdata        (cpu_wdata),
.fub_wstrb        (cpu_wstrb),
.fub_wvalid       (cpu_wvalid),
.fub_wready       (cpu_wready),

.fub_bresp        (cpu_bresp),
.fub_bvalid       (cpu_bvalid),
.fub_bready       (cpu_bready),

// Backend interface (to peripheral/memory)
.m_axil_awaddr    (periph_awaddr),
.m_axil_awprot    (periph_awprot),
.m_axil_awvalid   (periph_awvalid),
.m_axil_awready   (periph_awready),

.m_axil_wdata     (periph_wdata),
.m_axil_wstrb     (periph_wstrb),
.m_axil_wvalid    (periph_wvalid),
.m_axil_wready    (periph_wready),

.m_axil_bresp     (periph_bresp),
.m_axil_bvalid    (periph_bvalid),
.m_axil_bready    (periph_bready),

// Status
.busy             (wr_busy)
);

```

5.5.2 Register Write Example

```

// Write to control register at 0x2000
initial begin
    @(posedge aclk);

    // Present address and data simultaneously
    fub_awaddr  = 32'h2000;
    fub_awprot  = 3'b000;
    fub_awvalid = 1'b1;

    fub_wdata   = 32'hDEADBEEF;
    fub_wstrb   = 4'b1111; // All bytes valid
    fub_wvalid  = 1'b1;

    @(posedge aclk);
    while (!(fub_awready && fub_wready)) @(posedge aclk);

```

```

fub_awvalid = 1'b0;
fub_wvalid  = 1'b0;

// Wait for write response
fub_bready = 1'b1;
@(posedge aclk);
while (!fub_bvalid) @(posedge aclk);

$display("Write response: BRESP=%0d", fub_bresp);
end

```

5.5.3 Partial Write Example (Byte Strobes)

```

// Write only upper 16 bits to 32-bit register
fub_awaddr  = 32'h2004;
fub_awprot  = 3'b000;
fub_awvalid = 1'b1;

fub_wdata   = 32'hABCD0000; // Data in upper half
fub_wstrb   = 4'b1100;      // Only upper 2 bytes valid
fub_wvalid  = 1'b1;

// Result: Register[31:16] = 0xABCD, Register[15:0] unchanged

```

5.6 Design Notes

5.6.1 AXI4-Lite Protocol Constraints

Single-Beat Only: - No burst support (implicitly AWLEN=0, AWSIZE=log2(DW/8)) - Each AW+W transaction results in exactly 1 B response - No WLAST signal (always last and only beat)

No Out-of-Order: - No AWID/BID signals - Responses always in order - AW and W can arrive in any order (no dependency)

Reduced Signals: - No AWCACHE, AWQOS, AWREGION, AWLOCK, AWSIZE, AWLEN, AWBURST - Only AWADDR and AWPROT for control

5.6.2 Buffer Depth Selection

AW Channel (Addresses): - SKID_DEPTH_AW = 2 (4 entries): Typical for control registers - Increase if many back-to-back writes with slow slave

W Channel (Data): - SKID_DEPTH_W = 2 (4 entries): Matches AW depth - Should be >= SKID_DEPTH_AW to avoid W channel stalls

B Channel (Responses): - **SKID_DEPTH_B** = 2 (4 entries): Responses are single-beat - Smaller depth acceptable if frontend can always accept

5.6.3 Write Strobe (WSTRB) Usage

32-bit Data Width: |WSTRB[3:0] | Bytes Written | Use Case | |———|———|———| |
 4'b0001 | [7:0] | Byte write to offset 0 | | 4'b0010 | [15:8] | Byte write to offset 1 | | 4'b0011 |
 [15:0] | Half-word write | | 4'b1100 | [31:16] | Upper half-word | | 4'b1111 | [31:0] | Full word
 write |

64-bit Data Width: |WSTRB[7:0] | Bytes Written | Use Case | |———|———|———| |
 8'b0000_0001 | [7:0] | Byte write | | 8'b0000_1111 | [31:0] | Lower word | | 8'b1111_0000 |
 [63:32] | Upper word | | 8'b1111_1111 | [63:0] | Full double-word |

5.6.4 Busy Signal

Asserted when: - AW buffer has pending addresses (`int_aw_count > 0`) - W buffer has pending data (`int_w_count > 0`) - B buffer has pending responses (`int_b_count > 0`) - Frontend presenting new transaction (`fub_awvalid || fub_wvalid`) - Backend presenting response (`m_axil_bvalid`)

Use for: - Clock gating control (see `axil4_master_wr_cg`) - Power management - Interface idle detection

5.7 Performance Characteristics

5.7.1 Latency

Path	Cycles	Notes
Frontend → Backend (AW)	1	Skid buffer overhead
Frontend → Backend (W)	1	Skid buffer overhead
Backend → Frontend (B)	1	Skid buffer overhead
Total write latency	Slave latency + 3	AW + W + Slave + B

5.7.2 Throughput

Scenario	Rate	Notes
Back-to-back writes	1 write/cycle	If buffers not stalled
Typical peripheral	1 write/N cycles	N = slave response

Scenario	Rate	Notes
		time

5.7.3 Resource Usage

Resource	Count	Notes
LUTs	~300	Approximate (32-bit, 3 channels)
FFs	~250	Buffer state + control
BRAM	0	Uses distributed RAM

5.8 Related Modules

5.8.1 Core Modules

- [axil4_master_rd](#) - Read master counterpart
- [axil4_slave_rd](#) - Slave read interface
- [axil4_slave_wr](#) - Slave write interface

5.8.2 Monitor Modules

- [axil4_master_wr_mon](#) - Master write with monitoring

5.8.3 Clock-Gated Variants

- [axil4_master_wr_cg](#) - Clock-gated version

5.8.4 Supporting Infrastructure

- [gaxi_skid_buffer](#) - Elastic buffering (rtl/amba/gaxi/)
-

5.9 References

5.9.1 Specifications

- ARM IHI 0022E: AMBA AXI and ACE Protocol Specification
- Chapter 4: AXI4-Lite (simplified subset)

5.9.2 Documentation

- [RTLamba Overview](#) - Complete AMBA subsystem

- [AXI4 Master Write](#) - Full AXI4 comparison

5.9.3 Source Code

- RTL: rtl/amba/axil4/axil4_master_wr.sv
 - Tests: val/amba/test_axil4_master_wr.py
 - Framework: bin/TBClasses/components/axil4/
-

Last Updated: 2025-10-20

5.10 Navigation

- [← Back to AXIL4 Index](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

6 AXIL4 Slave Read Interface (Clock-Gated)

Module: axil4_slave_rd_cg.sv **Base Module:** [axil4_slave_rd](#) **Location:** rtl/amba/axil4/
Status: ✓ Production Ready

6.1 Quick Reference

This is the **clock-gated variant** of [axil4_slave_rd](#).

For complete clock-gating documentation, usage examples, and configuration guidelines, see:

→ [Clock-Gated Variants Guide](#)

6.2 Summary

The axil4_slave_rd_cg module adds power optimization to axil4_slave_rd through activity-based clock gating:

- ✓ **Same Functionality:** 100% equivalent to base module
- ✓ **Power Savings:** 25-70% depending on traffic utilization

- ✓ **Configurable:** Idle threshold, gating domains, enable/disable
 - ✓ **Zero Overhead When Disabled:** ENABLE_CLOCK_GATING=0 → identical to base
-

6.3 Common Parameters

In addition to all [axil4_slave_rd](#) parameters:

Parameter	Default	Description
ENABLE_CLOCK_GATING	1	Master enable (0=disable, identical to base)
CG_IDLE_CYCLES	8	Cycles before clock gating activates
CG_GATE_*	1	Domain-specific gating enables

6.4 Quick Usage

```
axil4_slave_rd_cg #(
    // Base module parameters (see axil4_slave_rd.md)
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),

    // Clock gating (see CG guide for details)
    .ENABLE_CLOCK_GATING(1),
    .CG_IDLE_CYCLES(8)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),
    // ... all other ports same as axil4_slave_rd
);
```

6.5 Documentation

- **Base Module Functionality:** [axil4_slave_rd.md](#)
- **Clock Gating Guide:** [clock_gated_variants.md](#)
- **Detailed CG Examples:**

- [axi4_master_rd_mon_cg.md](#) (AXI4 monitor)
 - [axil4_master_rd_mon_cg.md](#) (AXIL4 monitor)
 - [apb_slave_cg.md](#) (APB interface)
-

6.6 Navigation

- [← Back to AXIL4 Index](#)
- [← Back to RTLamba Index](#)
- [← Back to Main Documentation Index](#)

7 AXIL4 Slave Read

Module: axil4_slave_rd.sv **Location:** rtl/amba/axil4/ **Status:** ✓ Production Ready

7.1 Overview

The AXIL4 Slave Read module provides a buffered AXI4-Lite read interface for slave devices (memory, peripherals). It accepts read requests from an interconnect/master and forwards them to backend logic with elastic buffering for timing closure.

7.1.1 Key Features

- ✓ **AXI4-Lite Slave:** Buffered read-only interface (AR and R channels)
 - ✓ **Single-Beat Transactions:** No burst support
 - ✓ **Elastic Buffering:** Decouples interconnect and backend timing
 - ✓ **Activity Monitoring:** Busy signal for clock gating
 - ✓ **Minimal Latency:** 1-cycle buffer overhead per channel
-

7.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	Address bus width
AXIL_DATA_WIDTH	int	32	Data bus width (32 or 64)
SKID_DEPTH_AR	int	2	AR channel

Parameter	Type	Default	Description
SKID_DEPTH_R	int	4	buffer depth (2 ^N entries) R channel buffer depth (2 ^N entries)

7.3 Port Groups

7.3.1 Slave Interface (Input Side from Interconnect)

Read Address Channel (AR): | Port | Direction | Width | Description | |——|———|——|
 ————| | s_axil_araddr | Input | AW | Read address (from interconnect) | |
 s_axil_arprot | Input | 3 | Protection attributes | | s_axil_arvalid | Input | 1 | Address
 valid | | s_axil_arready | Output | 1 | Address ready (buffer status) |

Read Data Channel (R): | Port | Direction | Width | Description | |——|———|——|
 ————| | s_axil_rdata | Output | DW | Read data (to interconnect) | | s_axil_rresp |
 Output | 2 | Read response | | s_axil_rvalid | Output | 1 | Data valid | | s_axil_rready |
 Input | 1 | Data ready |

7.3.2 Backend Interface (Output Side to Memory/Logic)

Read Address Channel (AR): | Port | Direction | Width | Description | |——|———|——|
 ————| | fub_araddr | Output | AW | Read address (to backend) | | fub_arprot | Output | 3
 | Protection attributes | | fub_arvalid | Output | 1 | Address valid | | fub_arready | Input | 1
 | Address ready (from backend) |

Read Data Channel (R): | Port | Direction | Width | Description | |——|———|——|
 ————| | fub_rdata | Input | DW | Read data (from backend) | | fub_rresp | Input | 2 |
 Read response | | fub_rvalid | Input | 1 | Data valid | | fub_rready | Output | 1 | Data ready
 |

7.3.3 Status Outputs

Port	Direction	Width	Description
busy	Output	1	Interface active (for clock gating)

7.4 Usage Example

```
axil4_slave_rd #(
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AR(2),
    .SKID_DEPTH_R(4)
) u_axil_slave_rd (
    .aclk          (axi_clk),
    .aresetn       (axi_resetn),

    // Slave interface (from interconnect)
    .s_axil_araddr (interconnect_araddr),
    .s_axil_arprot (interconnect_arprot),
    .s_axil_arvalid (interconnect_arvalid),
    .s_axil_arready (interconnect_arready),

    .s_axil_rdata   (interconnect_rdata),
    .s_axil_rresp   (interconnect_rresp),
    .s_axil_rvalid  (interconnect_rvalid),
    .s_axil_rready  (interconnect_rready),

    // Backend interface (to memory/peripheral)
    .fub_araddr     (mem_araddr),
    .fub_arprot     (mem_arprot),
    .fub_arvalid    (mem_arvalid),
    .fub_arready    (mem_arready),

    .fub_rdata      (mem_rdata),
    .fub_rresp      (mem_rresp),
    .fub_rvalid     (mem_rvalid),
    .fub_rready     (mem_rready),

    // Status
    .busy           (rd_busy)
);
```

7.5 Design Notes

- **Signal Flow:** Interconnect → AR Buffer → Backend → R Buffer → Interconnect
- **Use Case:** Memory controllers, peripheral slaves, register blocks

- **Buffering:** Allows backend to respond at its own pace without stalling interconnect
-

7.6 Related Modules

- [axil4_slave_wr](#) - Slave write counterpart
 - [axil4_master_rd](#) - Master read interface
 - [axil4_slave_rd_mon](#) - Slave read with monitoring
 - [axil4_slave_rd_cg](#) - Clock-gated version
-

Last Updated: 2025-10-20

7.7 Navigation

- [← Back to AXIL4 Index](#)
- [← Back to RTLamba Index](#)
- [← Back to Main Documentation Index](#)

8 AXIL4 Slave Write Interface (Clock-Gated)

Module: `axil4_slave_wr_cg.sv` **Base Module:** [axil4_slave_wr](#) **Location:** `rtl/amba/axil4/`
Status: ✓ Production Ready

8.1 Quick Reference

This is the **clock-gated** variant of [axil4_slave_wr](#).

For complete clock-gating documentation, usage examples, and configuration guidelines, see:

→ [Clock-Gated Variants Guide](#)

8.2 Summary

The `axil4_slave_wr_cg` module adds power optimization to `axil4_slave_wr` through activity-based clock gating:

- ✓ **Same Functionality:** 100% equivalent to base module
 - ✓ **Power Savings:** 25-70% depending on traffic utilization
 - ✓ **Configurable:** Idle threshold, gating domains, enable/disable
 - ✓ **Zero Overhead When Disabled:** `ENABLE_CLOCK_GATING=0` → identical to base
-

8.3 Common Parameters

In addition to all `axil4_slave_wr` parameters:

Parameter	Default	Description
<code>ENABLE_CLOCK_GATING</code>	1	Master enable (0=disable, identical to base)
<code>CG_IDLE_CYCLES</code>	8	Cycles before clock gating activates
<code>CG_GATE_*</code>	1	Domain-specific gating enables

8.4 Quick Usage

```
axil4_slave_wr_cg #(
    // Base module parameters (see axil4_slave_wr.md)
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),

    // Clock gating (see CG guide for details)
    .ENABLE_CLOCK_GATING(1),
    .CG_IDLE_CYCLES(8)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),
    // ... all other ports same as axil4_slave_wr
);
```

8.5 Documentation

- **Base Module Functionality:** [axil4_slave_wr.md](#)
 - **Clock Gating Guide:** [clock_gated_variants.md](#)
 - **Detailed CG Examples:**
 - [axi4_master_rd_mon_cg.md](#) (AXI4 monitor)
 - [axil4_master_rd_mon_cg.md](#) (AXIL4 monitor)
 - [apb_slave_cg.md](#) (APB interface)
-

8.6 Navigation

- [← Back to AXIL4 Index](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

9 AXIL4 Slave Write

Module: `axil4_slave_wr.sv` **Location:** `rtl/amba/axil4/` **Status:** ✓ Production Ready

9.1 Overview

The AXIL4 Slave Write module provides a buffered AXI4-Lite write interface for slave devices (memory, peripherals). It accepts write requests from an interconnect/master and forwards them to backend logic with elastic buffering for timing closure.

9.1.1 Key Features

- ✓ **AXI4-Lite Slave:** Buffered write-only interface (AW, W, B channels)
 - ✓ **Single-Beat Transactions:** No burst support
 - ✓ **Byte Strobes:**WSTRB support for partial writes
 - ✓ **Elastic Buffering:** Decouples interconnect and backend timing
 - ✓ **Activity Monitoring:** Busy signal for clock gating
 - ✓ **Minimal Latency:** 1-cycle buffer overhead per channel
-

9.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	Address bus width
AXIL_DATA_WIDTH	int	32	Data bus width (32 or 64)
SKID_DEPTH_AW	int	2	AW channel buffer depth (2^N entries)
SKID_DEPTH_W	int	2	W channel buffer depth (2^N entries)
SKID_DEPTH_B	int	2	B channel buffer depth (2^N entries)

9.3 Port Groups

9.3.1 Slave Interface (Input Side from Interconnect)

Write Address Channel (AW): | Port | Direction | Width | Description | |——|———|——|
 ————| | s_axil_awaddr | Input | AW | Write address (from interconnect) | |
 s_axil_awprot | Input | 3 | Protection attributes | | s_axil_awvalid | Input | 1 | Address
 valid | | s_axil_awready | Output | 1 | Address ready (buffer status) |

Write Data Channel (W): | Port | Direction | Width | Description | |——|———|——|
 ————| | s_axil_wdata | Input | DW | Write data (from interconnect) | | s_axil_wstrb |
 Input | DW/8 | Write strobes | | s_axil_wvalid | Input | 1 | Data valid | | s_axil_wready |
 Output | 1 | Data ready (buffer status) |

Write Response Channel (B): | Port | Direction | Width | Description | |——|———|——|
 ————| | s_axil_bresp | Output | 2 | Write response (to interconnect) | | s_axil_bvalid |
 Output | 1 | Response valid | | s_axil_bready | Input | 1 | Response ready |

9.3.2 Backend Interface (Output Side to Memory/Logic)

Write Address Channel (AW): | Port | Direction | Width | Description | |——|———|——|
 ————| | fub_awaddr | Output | AW | Write address (to backend) | | fub_awprot | Output |
 3 | Protection attributes | | fub_awvalid | Output | 1 | Address valid | | fub_awready | Input |
 1 | Address ready (from backend) |

Write Data Channel (W): | Port | Direction | Width | Description | |——|———|——|
 ————| | fub_wdata | Output | DW | Write data (to backend) | | fub_wstrb | Output | DW/8
 | Write strobes | | fub_wvalid | Output | 1 | Data valid | | fub_wready | Input | 1 | Data ready
 (from backend) |

Write Response Channel (B): | Port | Direction | Width | Description | |——|———|——|
 ————| | fub_bresp | Input | 2 | Write response (from backend) | | fub_bvalid | Input | 1 |
 Response valid | | fub_bready | Output | 1 | Response ready |

9.3.3 Status Outputs

Port	Direction	Width	Description
busy	Output	1	Interface active (for clock gating)

9.4 Usage Example

```
axil4_slave_wr #(
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AW(2),
    .SKID_DEPTH_W(2),
    .SKID_DEPTH_B(2)
) u_axil_slave_wr (
    .aclk          (axi_clk),
    .aresetn       (axi_resetn),

    // Slave interface (from interconnect)
    .s_axil_awaddr (interconnect_awaddr),
    .s_axil_awprot (interconnect_awprot),
    .s_axil_awvalid (interconnect_awvalid),
    .s_axil_awready (interconnect_awready),

    .s_axil_wdata  (interconnect_wdata),
    .s_axil_wstrb  (interconnect_wstrb),
    .s_axil_wvalid (interconnect_wvalid),
    .s_axil_wready (interconnect_wready),

    .s_axil_bresp  (interconnect_bresp),
    .s_axil_bvalid (interconnect_bvalid),
    .s_axil_bready (interconnect_bready),

    // Backend interface (to memory/peripheral)
    .fub_awaddr    (mem_awaddr),
```

```

.fub_awprot      (mem_awprot),
.fub_awvalid     (mem_awvalid),
.fub_awready     (mem_awready),

.fub_wdata       (mem_wdata),
.fub_wstrb       (mem_wstrb),
.fub_wvalid      (mem_wvalid),
.fub_wready      (mem_wready),

.fub_bresp       (mem_bresp),
.fub_bvalid      (mem_bvalid),
.fub_bready      (mem_bready),

// Status
.busy            (wr_busy)
);

```

9.5 Design Notes

- **Signal Flow:** Interconnect → AW/W Buffers → Backend → B Buffer → Interconnect
 - **Use Case:** Memory controllers, peripheral slaves, register blocks
 - **Buffering:** Allows backend to respond at its own pace without stalling interconnect
 - **Write Strobes:** Backend must honor WSTRB for partial word writes
-

9.6 Related Modules

- [axil4_slave_rd](#) - Slave read counterpart
 - [axil4_master_wr](#) - Master write interface
 - [axil4_slave_wr_mon](#) - Slave write with monitoring
 - [axil4_slave_wr_cg](#) - Clock-gated version
-

Last Updated: 2025-10-20

9.7 Navigation

- [← Back to AXIL4 Index](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

10 AXIL4 (AXI4-Lite) Modules

Location: rtl/amba/axil4/ **Test Location:** val/amba/ **Status:** ✓ Production Ready

10.1 Overview

The AXIL4 subsystem provides a complete implementation of the ARM AMBA AXI4-Lite protocol—a simplified subset of AXI4 designed for control register access with reduced signal count and single-beat transactions.

10.1.1 Key Features

- ✓ **AXI4-Lite Protocol:** Simplified subset (no burst, no ID, no QoS)
- ✓ **Single-Beat Transactions:** Always 1 transfer per transaction
- ✓ **Reduced Signals:** 40-50% fewer signals vs full AXI4
- ✓ **Buffered Interfaces:** Elastic buffering via gaxi_skid_buffer
- ✓ **Monitoring Infrastructure:** Integrated transaction monitoring
- ✓ **Clock Gating Variants:** Power-optimized versions
- ✓ **Typical Use:** Control registers, CSRs, peripheral access

10.2 Module Categories

10.2.1 Core Master/Slave Modules

Module | Description | Documentation | Status |

axil4_master_rd AXIL4 read master with buffered channels axil4_master_rd.md ✓ Documented	axil4_master_wr AXIL4 write master with buffered channels axil4_master_wr.md ✓ Documented	axil4_slave_rd AXIL4 read slave with buffered channels axil4_slave_rd.md ✓ Documented	axil4_slave_wr AXIL4 write slave with buffered channels axil4_slave_wr.md ✓ Documented

10.2.2 Monitor Modules (Verification)

Module	Description	Documentation	Status
axil4_master_rd_mon	Read master with integrated monitoring	axil4_master_rd_mon.md	✓ Documented
axil4_master_wr_mon	Write master with integrated monitoring	axil4_master_wr_mon.md	✓ Documented
axil4_slave_rd_mon	Read slave with integrated monitoring	axil4_slave_rd_mon.md	✓ Documented
axil4_slave_wr_mon	Write slave with integrated monitoring	axil4_slave_wr_mon.md	✓ Documented

10.2.3 Clock-Gated Variants

All clock-gated variants documented in: [axil4_clock_gating_guide.md](#)

Module	Base Module	Status
axil4_master_rd_cg	axil4_master_rd	✓ Documented
axil4_master_wr_cg	axil4_master_wr	✓ Documented
axil4_slave_rd_cg	axil4_slave_rd	✓ Documented
axil4_slave_wr_cg	axil4_slave_wr	✓ Documented
axil4_master_rd_mon_cg	axil4_master_rd_mon	✓ Documented
axil4_master_wr_mon_cg	axil4_master_wr_mon	✓ Documented
axil4_slave_rd_mon_cg	axil4_slave_rd_mon	✓ Documented
axil4_slave_wr_mon_cg	axil4_slave_wr_mon	✓ Documented

10.3 AXI4-Lite Protocol Overview

10.3.1 Simplified vs Full AXI4

Feature	AXI4	AXI4-Lite
Burst Support	Yes (1-256 beats)	No (always 1 beat)

Feature	AXI4	AXI4-Lite
ID Signals	ARID/AWID/RID/BID	None
Out-of-Order	Supported via ID	Not supported
Data Width	8-1024 bits	32 or 64 bits
QoS/Region	Supported	Not supported
Lock/Cache	Full support	AWLOCK/ARLOCK only
Signal Count	~40-50 signals	~20-25 signals
Typical Use	High-bandwidth data	Control registers

10.3.2 Channel Architecture

AXI4-Lite uses five independent channels for read and write transactions:

Write Transaction: 1. **AW (Write Address)** - Address and protection 2. **W (Write Data)** - Data and byte strobes 3. **B (Write Response)** - Completion status

Read Transaction: 1. **AR (Read Address)** - Address and protection 2. **R (Read Data)** - Data and response

10.3.3 Key Signals (Simplified)

Address Channels (AR/AW): - ARADDR/AWADDR - Byte address - ARPROT/AWPROT - Protection attributes (3 bits) - ARVALID/AWVALID, ARREADY/AWREADY - Handshake

Data Channels (R/W): - RDATA/WDATA - Transfer data (32 or 64 bits) - RRESP/BRESP - Response (OKAY, SLVERR, DECERR) - WSTRB - Write strobes (byte lane enables) - RVALID/WVALID, RREADY/WREADY - Handshake

10.4 Quick Start

10.4.1 Basic Read Master

```
axil4_master_rd #(
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AR(2),      // 4 entries
    .SKID_DEPTH_R(4)       // 16 entries
) u_rd_master (
    .aclk                (axi_clk),
    .aresetn              (axi_resetn),
```



```

// Frontend interface (from CPU)
.fub_araddr      (cpu_araddr),
.fub_arprot      (cpu_arprot),
.fub_arvalid     (cpu_arvalid),
.fub_arready     (cpu_arready),

.fub_rdata       (cpu_rdata),
.fub_rresp       (cpu_rresp),
.fub_rvalid      (cpu_rvalid),
.fub_rready      (cpu_rready),

// Master interface (to interconnect)
.m_axil_araddr   (periph_araddr),
// ... rest of AR/R signals

.busy            (rd_busy)
);

```

10.4.2 Basic Write Slave

```

axil4_slave_wr #(
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AW(2),
    .SKID_DEPTH_W(2),
    .SKID_DEPTH_B(2)
) u_wr_slave (
    .aclk          (axi_clk),
    .aresetn       (axi_resetn),

    // Slave interface (from interconnect)
    .s_axil_awaddr (interconnect_awaddr),
    // ... rest of AW/W/B signals

    // Backend interface (to register block)
    .fub_awaddr    (regfile_awaddr),
    // ... rest of AW/W/B signals

    .busy          (wr_busy)
);

```

10.4.3 Monitor Integration

```

axil4_master_rd_mon #(
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .UNIT_ID(1),

```

```

        .AGENT_ID(10),
        .MAX_TRANSACTIONS(8),      // Reduced for AXIL
        .ENABLE_FILTERING(1)
    ) u_rd_mon (
        .aclk                      (axi_clk),
        .aresetn                   (axi_resetn),

        // AXIL interfaces
        .fub_axil_araddr           (cpu_araddr),
        // ... rest of AR/R signals

        // Monitor configuration
        .cfg_monitor_enable        (1'b1),
        .cfg_error_enable          (1'b1),
        .cfg_timeout_enable        (1'b1),
        .cfg_perf_enable           (1'b0), // Avoid congestion
        .cfg_timeout_cycles        (16'd1000),

        // Filtering masks
        .cfg_axi_pkt_mask          (16'b1111_1111_0000_0011),
        // ... rest of masks

        // Monitor bus output
        .monbus_pkt_valid          (mon_valid),
        .monbus_pkt_ready          (mon_ready),
        .monbus_pkt_data           (mon_packet),

        // Status
        .busy                      (rd_busy),
        .active_transactions       (rd_active),
        .error_count               (rd_errors)
    );

```

10.5 Testing

All AXIL4 modules are verified using CocoTB-based testbenches:

Run all AXIL4 tests

```
pytest val/amba/test_axil4*.py -v
```

Run specific module tests

```
pytest val/amba/test_axil4_master_rd.py -v
```

```
pytest val/amba/test_axil4_slave_wr.py -v
```

```
pytest val/amba/test_axil4_master_rd_mon.py -v
```

```
# Run with waveforms
```

```
pytest val/amba/test_axil4_master_rd.py --vcd=waves.vcd -v
```

10.6 Design Patterns

10.6.1 Pattern 1: Buffered Master/Slave

All core modules use `gaxi_skid_buffer`: - Decouples frontend and backend timing - Configurable depth per channel - 1-cycle latency overhead - Full backpressure handling

10.6.2 Pattern 2: Monitor Integration

Monitor modules combine functional core with verification: - Uses shared **`axi_monitor_filtered`** (`rtl/amba/shared/`) - 3-level filtering hierarchy - 128-bit monitor bus + 64-bit side-band timestamp - Simplified for AXIL (`MAX_TRANSACTIONS=8`)

10.6.3 Pattern 3: Clock Gating

Clock-gated variants (`*_cg`) add power management: - Dynamic gating based on busy signal - Test mode support - 0-cycle ungating latency - Better power savings than AXI4 (bursty register access)

10.7 Performance Characteristics

10.7.1 Throughput

Configuration	Rate	Notes
Single-beat read/write	1 txn/cycle	When buffers not stalled
Typical peripheral	1 txn/N cycles	N = backend response time

10.7.2 Latency

Path	Cycles	Notes
AR → R (single)	2-3	Minimum read latency
AW+W → B (single)	2-3	Minimum write latency
Buffer overhead	+1 per channel	Skid buffer latency

Path	Cycles	Notes
------	--------	-------

10.7.3 Resource Usage

Module Type	LUTs	FFs	BRAM	Notes
Master/ Slave (32-bit)	~200	~150	0	Per direction (rd/wr)
Monitor (+mon)	+800	+600	0	Additional overhead
Clock-gated (+cg)	+50	+30	0	Clock gating logic

vs AXI4: ~40-50% resource savings (no ID tracking, no burst, simpler protocol)

10.8 Related Documentation

10.8.1 Protocol Specifications

- ARM IHI 0022E: AMBA AXI Protocol Specification
- Chapter 4: AXI4-Lite

10.8.2 RTL Design Sherpa Documentation

- [RTLamba Overview](#) - Complete AMBA subsystem architecture
- [AXI4 Modules](#) - Full AXI4 protocol (comparison)
- [APB Modules](#) - Even simpler peripheral bus
- [GAXI Modules](#) - Generic AXI utilities (buffers, FIFOs)

10.8.3 Configuration and Integration

- [AXI Monitor Configuration Guide](#) - Monitor setup strategies
- [Monitor Packet Specification](#) - 64-bit packet format

10.8.4 Source Code

- RTL: rtl/amba/axil4/
- Tests: val/amba/test_axil4*.py
- Framework: bin/TBClasses/components/axil4/
- Shared Infrastructure: rtl/amba/shared/ (monitor base, gaxi_skid_buffer)

10.9 Design Notes

10.9.1 When to Use AXI4-Lite

✓ **Use AXI4-Lite For:** - Control and status registers (CSRs) - Peripheral configuration interfaces - Low-bandwidth control paths - Resource-constrained designs - Simpler protocol requirements

✗ **Use Full AXI4 For:** - High-bandwidth data transfers - Burst transactions required - Out-of-order support needed - DMA engines, memory controllers - Streaming data

10.9.2 Buffer Depth Guidelines

Address Channels (AR/AW): - Default: 2 (4 entries) - sufficient for most register access - Increase for high-latency address decode

Data Channels (R/W): - Default: 4 (16 entries) for R, 2 (4 entries) for W - R channel typically deeper (accommodate backend latency) - W channel can be shallower (single-beat)

Response Channel (B): - Default: 2 (4 entries) - responses are single-beat - Rarely needs adjustment

Last Updated: 2025-10-20

10.10 Navigation

- [← Back to RTLamba Index](#)
- [← Back to Main Documentation Index](#)